

Ir Além 2 — Automação Inteligente com RPA, IA e Dados Híbridos

Robô de monitoramento cardiológico com bancos relacional (SQLite) e não relacional (MongoDB)

Curso de Tecnologia em Inteligência Artificial — FIAP | Disciplina: AIRPA

1. Objetivo

Este relatório descreve a extensão “Ir Além 2” do Assistente Cardiológico: um robô de RPA que monitora periodicamente sinais vitais simulados (pressão arterial, frequência cardíaca e adesão ao tratamento), interpreta mensagens de texto livre de pacientes e aplica técnicas simples de IA para identificar situações que demandam atenção — registrando cada decisão de forma rastreável em um banco não relacional. A solução combina um banco de dados relacional (SQLite), usado como fonte de verdade estruturada, com um banco de dados não relacional (MongoDB), usado como trilha de auditoria e repositório de conteúdo textual/variável, conforme os conceitos de hyperautomação apresentados na disciplina.

2. Continuidade do projeto

O Assistente Cardiológico já passou por duas etapas anteriores, e esta terceira etapa reaproveita deliberadamente o mesmo critério clínico de gravidade definido desde a Parte 1 (dor no peito, falta de ar, dor no braço e suor frio como sinais de alerta), agora aplicado em um contexto de monitoramento contínuo e automatizado:

Etapa	O que foi feito	Papel da IA
Parte 1	Assistente conversacional (watsonx Assistant + Flask) para triagem inicial de sintomas relatados pelo próprio paciente.	NLP de baixo código (intents/entities do watsonx Assistant)
Ir Além 1	Extração de sinais de alerta a partir de texto clínico livre, usando Chain of Thought (Ollama, llama3.1:8b).	LLM com raciocínio explícito (CoT)
Ir Além 2 (este)	Robô de RPA que monitora sinais vitais periódicos e mensagens de texto de forma contínua, sem intervenção humana a cada ciclo.	Regras fixas + IsolationForest + casamento de palavras-chave

A diferença central em relação às etapas anteriores é o regime de execução: as Partes 1 e Ir Além 1 respondem a uma interação pontual do paciente; o Ir Além 2 roda em ciclos periódicos, avaliando dados que se acumulam entre uma execução e outra — por isso a ênfase em rastreabilidade (cada alerta precisa ser explicável mesmo sem um humano presente no momento em que foi gerado).

3. Arquitetura da solução

A automação é organizada em três componentes:

- SQLite (relacional): armazena o cadastro de pacientes e as leituras de sinais vitais — a fonte de verdade estruturada, com colunas de controle que o próprio robô preenche (processado_robô, alerta_regra, anomalia_ia, anomalia_score).
- Robô de RPA (Python): script que roda em ciclos (uma vez ou em loop com intervalo configurável), lendo os dois bancos, aplicando as técnicas de IA e gravando os resultados.
- MongoDB (não relacional): guarda logs de execução do robô, mensagens de texto de pacientes e os alertas gerados — dados de formato mais variável e cujo papel é auditoria/rastreabilidade, não a fonte de verdade clínica.

O fluxo completo de cada ciclo está implementado em `src/robo_monitoramento.py::executar_ciclo()` e segue estes passos:

1. Lê mensagens de texto pendentes no MongoDB e interpreta com casamento de palavras-chave.
2. Lê leituras de sinais vitais pendentes no SQLite (processado_robô = 0).
3. Aplica regras fixas de limiar clínico sobre essas leituras.
4. Treina/aplica um IsolationForest sobre o histórico para detectar anomalias multivariadas.
5. Grava todo alerta gerado — de qualquer origem — na coleção alertas do MongoDB, com o `execucao_id` do ciclo.
6. Persiste os rótulos de anomalia de volta no SQLite (governança, mesmo padrão da tabela `validacao_execucoes` discutida em aula).
7. Grava um log de execução (metadados do processo) no MongoDB.

4. Banco de dados relacional (SQLite)

O SQLite foi escolhido pelos mesmos motivos discutidos em aula: é embedded, zero-configuration, e adequado como “banco de controle” local de uma automação — não é necessário provisionar um servidor separado para o projeto. As garantias ACID das suas transações são relevantes aqui porque múltiplas leituras podem ser atualizadas na mesma execução do robô sem risco de deixar o banco em estado inconsistente caso o processo seja interrompido no meio (o código também usa rollback explícito em caso de erro).

4.1 Tabela pacientes

Coluna	Descrição	Tipo
id	chave primária	INTEGER (PK)
nome, idade	identificação básica	TEXT, INTEGER
fatores_risco	texto livre, mesmo padrão da Parte 1	TEXT

4.2 Tabela sinais_vitais

É a tabela sobre a qual o robô atua. As quatro últimas colunas não são preenchidas na inserção — são o “cartão de controle” que o robô atualiza a cada ciclo, permitindo que a próxima execução saiba exatamente o que já foi avaliado:

Coluna	Descrição	Origem
pressao_sistolica / diastolica, frequencia_cardiaca, adesao_tratamento_pct	sinais vitais simulados periodicamente	simulação
processado_robo	0/1 — se este ciclo já avaliou a leitura	robô
alerta_regra	0/1 — se algum limiar fixo foi violado	robô (regras)
anomalia_ia, anomalia_score	rótulo e score do IsolationForest	robô (IA)

A chave estrangeira `paciente_id` garante a integridade referencial entre as duas tabelas; os índices em (`processado_robo`) e (`paciente_id`, `timestamp`) existem especificamente para que o robô não precise varrer a tabela inteira a cada ciclo — apenas as leituras pendentes. O DDL completo está em `db/schema_sqlite.sql`.

5. Banco de dados não relacional (MongoDB)

O MongoDB foi usado, local, via Compass, para as três coleções descritas abaixo. A justificativa de projeto é a mesma discutida na aula sobre bancos schemaless: os documentos aqui têm formato naturalmente variável — uma mensagem de paciente não tem a mesma estrutura de um log de execução, e um alerta muda de forma dependendo da sua origem (um alerta por regra fixa não tem `score_anomalia`; um alerta por IsolationForest tem). Forçar isso em tabelas relacionais rígidas exigiria várias colunas nulas ou tabelas separadas por origem — o document store absorve essa variação naturalmente.

Coleção	Conteúdo	Chave de rastreio
logs_execucao	um documento por ciclo do robô: início, fim, duração, contagens, status	execucao_id
mensagens_pacientes	mensagens de texto livre simuladas e seu resultado da interpretação por palavra-chave	paciente_id
alertas	todo alerta gerado, de qualquer origem, com snapshot dos dados que o motivaram	execucao_id + paciente_id

O detalhamento de cada documento (campos, tipos, exemplo em JSON) está em `db/mongo_schema.md`; os índices de apoio são criados de forma idempotente por `db/mongo_setup.py`. Vale registrar o trade-off consciente: o MongoDB aqui roda em uma única instância local (sem sharding/replicação), então a discussão do Teorema CAP se aplica apenas em espírito — o ganho real, neste projeto, é a flexibilidade de esquema para dados de auditoria, não a disponibilidade distribuída.

6. Técnicas de IA aplicadas

6.1 Regras fixas (baseline determinístico)

Antes de qualquer modelo estatístico, o robô aplica limiares clínicos simples (ex.: pressão sistólica ≥ 180 ou ≤ 90 mmHg, frequência cardíaca ≥ 120 ou ≤ 40 bpm, adesão ao tratamento $\leq 50\%$) — a mesma lógica condicional da Action “Relatar sintoma” da Parte 1, adaptada de sintomas categóricos para valores numéricos contínuos. Essas regras existem porque nem todo caso crítico precisa de um modelo estatístico para ser identificado: um pico hipertensivo isolado é óbvio por definição, e tratá-lo com regra fixa é mais rápido, mais barato e mais auditável do que depender de um modelo.

6.2 IsolationForest (detecção de anomalias multivariadas)

As regras fixas não capturam combinações atípicas de valores que, isoladamente, estão dentro da faixa aceitável — por exemplo, pressão no limite superior da normalidade associada a frequência cardíaca também no limite e adesão ao tratamento caindo. Para esse tipo de padrão, foi aplicado o IsolationForest (scikit-learn), seguindo exatamente o pipeline apresentado em aula: o algoritmo constrói árvores de partição aleatória, e observações que exigem menos cortes para serem isoladas recebem rótulo de anomalia. Foi usado `contamination=0.08`, o mesmo valor do exemplo da aula, assumindo que cerca de 8% do histórico de leituras representa desvios legítimos a investigar. O modelo é re-treinado a cada ciclo com todo o histórico disponível (não apenas o lote novo), o que dá uma base mais estável do que é “normal” para o conjunto de pacientes monitorado; os rótulos e o score (decision_function) são então persistidos de volta na tabela `sinais_vitais` — a mesma prática de governança descrita em aula para a tabela `validacao_execucoes`.

6.3 Interpretação de texto por palavras-chave e conexão com o Ir Além 1

Mensagens de texto de pacientes são avaliadas por casamento de palavras-chave contra a mesma lista de termos de alerta usada nas etapas anteriores. Essa escolha foi deliberada: o robô roda em ciclos curtos e frequentes, e chamar um LLM local (como o Ollama usado no Ir Além 1 com Chain of Thought) a cada execução seria mais lento e mais custoso computacionalmente do que o necessário para uma triagem de primeira camada. O papel desta etapa é sinalizar candidatos; uma mensagem marcada aqui é justamente o tipo de caso que se beneficiaria da análise mais profunda e auditável do Chain of Thought, se um humano decidir investigar — as duas técnicas são complementares, não concorrentes, e cada uma foi aplicada onde seu custo-benefício faz mais sentido.

7. Rastreabilidade e auditoria

Cada execução do robô recebe um `execucao_id` (UUID) gerado no início do ciclo. Todo alerta produzido naquele ciclo — seja por regra fixa, IsolationForest ou mensagem de texto — carrega esse mesmo `execucao_id`, além de um snapshot dos dados brutos que o motivaram. Isso permite reconstruir, a qualquer momento, três perguntas de auditoria: quais alertas um determinado ciclo gerou, o que motivou um alerta específico, e se aquele ciclo terminou com sucesso ou erro (campo `status/detalhes_erro` em `logs_execucao`, com rollback automático da transação SQLite em caso de falha).

8. Resultados observados

Em uma execução real do pipeline (dados simulados de 5 pacientes, 290 leituras históricas usadas para treinar o IsolationForest e um lote de 12 leituras + 8 mensagens pendentes), o robô produziu 8 alertas nesse único ciclo:

Origem	Alertas gerados	Exemplo real de motivo registrado
mensagem_texto	4	Termo “aperto no peito” identificado no relato do paciente
regra_fixa	2	Pressão sistólica 190 mmHg; frequência cardíaca 135 bpm
isolation_forest	2	Combinação atípica de pressão/frequência/adesão, sem violar nenhum limiar fixo isoladamente

Uma segunda execução imediatamente após a primeira, sem novos dados inseridos, avaliou corretamente 0 leituras e 0 mensagens novas (nenhum alerta duplicado) — confirmando que o controle processado_robo evita reprocessamento e, portanto, alertas repetidos para o mesmo dado.

9. Limitações e melhorias futuras

- Os dados clínicos são inteiramente simulados; não há validação com dados reais de pacientes.
- O IsolationForest é re-treinado do zero a cada ciclo sobre todo o histórico — funciona bem na escala deste projeto, mas não escalaria indefinidamente sem uma estratégia de re-treino incremental ou por janela de tempo.
- A interpretação de texto por palavras-chave não captura sinônimos ou reformulações fora da lista de termos — uma extensão natural seria acionar o pipeline de Chain of Thought (Ir Além 1) automaticamente para toda mensagem sinalizada, em vez de exigir revisão humana manual.
- O MongoDB roda em instância única local; um ambiente de produção real avaliaria replicação para disponibilidade, conforme o Teorema CAP discutido em aula.

10. Conclusão

O projeto demonstrou a integração entre RPA, IA e dados híbridos proposta pela disciplina: um robô Python executa ciclos periódicos que leem um banco relacional (fonte de verdade estruturada dos sinais vitais) e um banco não relacional (trilha de auditoria e conteúdo textual variável), aplicando duas técnicas de IA complementares — regras fixas para casos óbvios e IsolationForest para anomalias multivariadas — além de uma camada leve de interpretação textual. A decisão de projeto mais relevante foi a divisão de responsabilidade entre os dois bancos: dados estruturados e sujeitos a controle transacional ficam no SQLite; logs, mensagens e alertas — cujo formato varia por natureza e cujo papel é rastreabilidade, não verdade clínica — ficam no MongoDB. Essa separação, mais do que uma exigência técnica do enunciado, reflete o mesmo raciocínio de arquitetura discutido em aula: usar cada modelo de banco de dados onde suas garantias específicas realmente importam.